

SPARC Roadmap — Agent-Driven SDLC

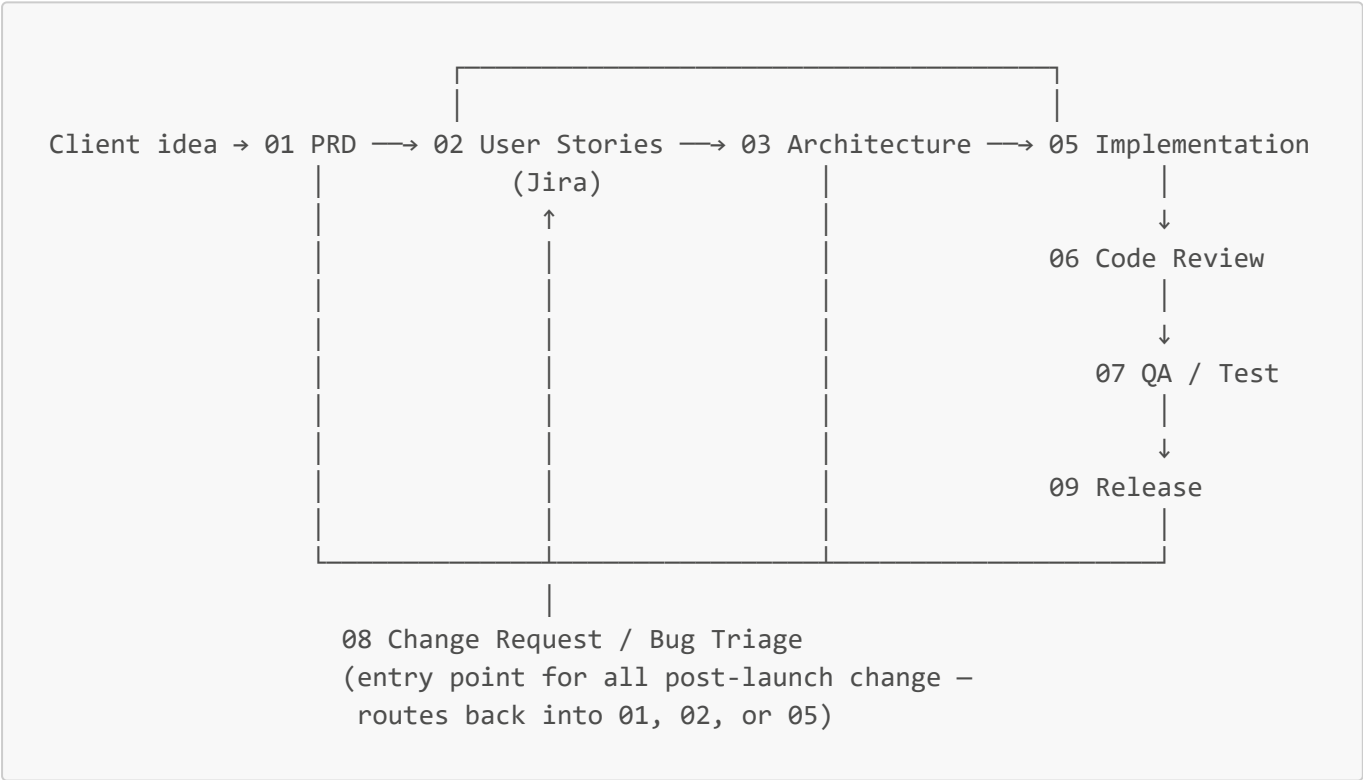
Purpose of this file: a self-contained handoff. If you are an AI or a person picking this project up in a fresh session with no prior context, read this file first. It tells you what SPARC is, what already exists, the rules every agent must follow, and what to build next.

Last updated: 2026-09-13

1. What SPARC is

We are modelling how a software company actually works. A client arrives with an idea; it passes through specialists — PM, BA, architect, developer, QA, support — and each hands a document to the next. We replace each specialist with an agent.

The pipeline is not just a forward document generator. Its real purpose is to survive **change**: a bug report or change request arriving months later must be traceable back to the requirement that caused the work. Everything in the schema exists to serve that.



2. Current state

#	Agent	Status	Location
01	PRD Agent	Built	.claude/agents/prd-agent.md
02	User Stories Agent	Built	.claude/agents/user-stories-agent.md

#	Agent	Status	Location
03	Architecture Agent	Built (not yet run — OQ-4 gates half its output)	.claude/agents/architecture-agent.md
04–09	everything else	Not built	—

Live artifacts:

- PRD: [1. PRD/prd-premium-perfume-ecommerce.md](#) — v1.1, Status: Confirmed, REQ-001–REQ-015
- Backlog: Jira project **ACRI** on [experionglobal.atlassian.net](#) (cloudId [69a5faff-afbd-40dc-ac75-1b2a52db5362](#))
 - Epics [ACRI-3–ACRI-7](#), Stories [ACRI-8–ACRI-28](#) (US-001–US-021)
- Local index: [2. User Stories/story-index.md](#)

Jira connection: already authenticated via the account-level claude.ai Atlassian Rovo connector. Not a project [.mcp.json](#) — nothing to configure locally.

Known open items:

- **REQ-009** (order tracking) has no story — blocked by **OQ-1** (courier partner not chosen).
- **PRD Non-Functional bucket is empty (OQ-4)** — no story carries a performance/security/accessibility/data-retention criterion. This blocks sound architecture work; resolve before or alongside agent 03.
- [Depends-On](#) relationships exist as description text but not as real Jira issue links.

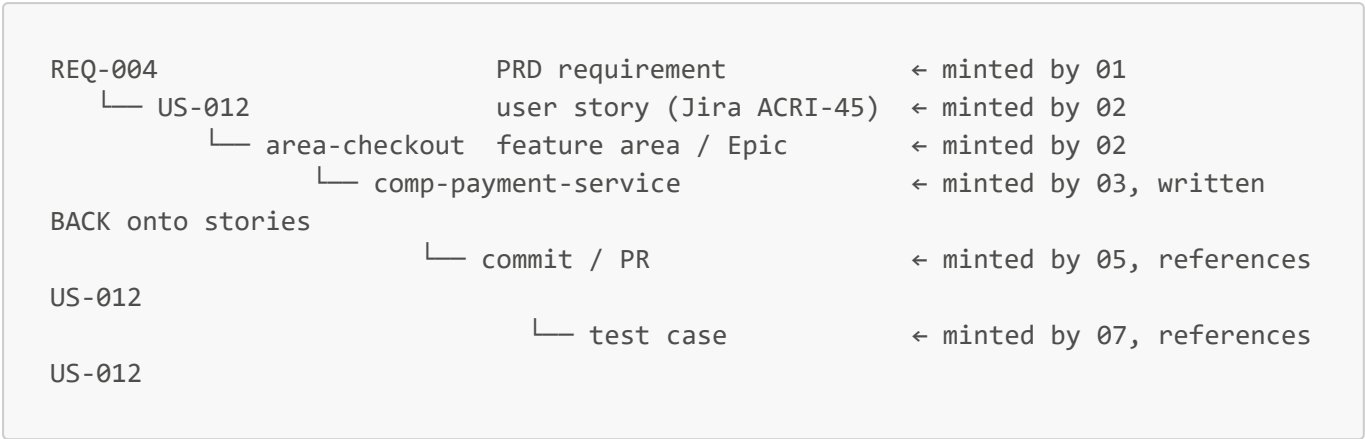
3. Non-negotiable rules for every agent

Full reasoning in [agents/AGENT-DESIGN-GUIDE.md](#). The short version — violate these and the pipeline stops working:

1. **One agent = one transformation.** One input document, one output artifact. An agent that "also does a bit of the next stage" makes that stage un-overridable.
2. **Ids are permanent.** [REQ-004](#), [US-012](#) mean the same thing forever. Never renumber, never reuse a retired id. Things are **retired or closed, never deleted**.
3. **Labels are the retrieval axis, not Jira Components.** Jira rejects component names that don't already exist and there's no MCP tool to create them. Labels are free-form and exact-matched in JQL. Lowercase kebab-case, no spaces.
4. **Idempotent re-runs.** Search before you create. A re-run against unchanged input creates nothing and edits nothing. Scope re-syncs from the source document's Change Log.
5. **Approval gate before shared/irreversible writes.** Draft locally, show a summary, wait for explicit human approval before writing to Jira/git/prod. Local and reversible → go ahead.
6. **Vocabulary is inherited, not invented.** Use the PRD Glossary's canonical terms and persona slugs everywhere. Synonym drift silently breaks [text](#) ~ retrieval.
7. **Report your gaps.** Coverage holes, blocked items, unreferenced NFRs. An agent that hides gaps is worse than one that has them.
8. **Verify tool schemas before writing a prompt.** Assumptions about external APIs belong in the "check first" pile.

4. The traceability chain

This is the spine. Every agent either inherits a link or mints one.



Label schema on every story (exactly these four kinds, nothing else):

Label	Example	Answers
us-<nnn>	us-012	Which story is this?
req-<nnn>	req-004	Which requirement does it implement? (repeatable)
area-<slug>	area-checkout	Which feature area / Epic?
phase-<bucket>	phase-v1	In scope now?

Plus `comp-<module>` added by agent 03.

The queries this enables — these are the reason for the schema:

Situation	JQL
Bug in checkout mentioning "coupon"	<code>project = ACRI AND labels = area-checkout AND text ~ "coupon"</code>
CR against a requirement	<code>project = ACRI AND labels = req-004</code>
Fetch one known story	<code>project = ACRI AND labels = us-012</code>
All v1 scope	<code>project = ACRI AND labels = phase-v1</code>
Everything touching a module	<code>project = ACRI AND labels = comp-payment-service</code>

5. Agent roster

01 — PRD Agent  built

Replaces	PM/BA doing client intake
Input	Client conversation (raw idea, brief, or amendment request)

Output	1. PRD/prd-<name>.md — versioned, Status Draft/Confirmed, numbered REQ-xxx, Glossary, persona slugs, Open Questions with Blocks:, Change Log
Mints	REQ-xxx ids, glossary vocabulary, persona slugs, OQ-n
Feeds	02 (primary), 03 (NFRs + constraints), 08 (CRs amend the PRD)
Does not	Design solutions, pick tech, estimate, write tickets

02 — User Stories Agent built

Replaces	BA turning a signed-off PRD into a sprint backlog
Input	Confirmed PRD (refuses Draft)
Output	Jira Epics + Stories in ACRI; 2. User Stories/backlog-draft.md; story-index.md cache
Mints	US-xxx ids, area-<slug> feature areas
Inherits	REQ-xxx, glossary, persona slugs, OQ-n blocks
Feeds	03 (areas → modules), 05 (stories → work), 07 (AC → test cases), 08 (target of CR lookups)
Does not	Pick tech, design schemas, estimate, delete tickets

03 — Architecture Agent built

Replaces	Tech lead / solution architect
Input	Confirmed PRD (esp. Non-Functional + Constraints) + Jira backlog
Output	3. Architecture/architecture.md — stack choice with rationale, module/service boundaries, data model, API surface, integration points, NFR strategy, ADRs
Mints	comp-<module> labels — and must write them back onto the Jira stories , closing the chain
Inherits	area-* (as the starting shape for modules), REQ-* incl. NFRs
Feeds	05 (what to build and where), 06 (review standards), 09 (deploy topology)
Does not	Write implementation code, create tickets
Blocked by	OQ-4 — you cannot choose a stack sensibly with zero NFRs. Resolve first.

Key design notes: every architectural decision cites the REQ-/NFR id driving it. Areas from 02 are a *starting hypothesis* for module boundaries, not a mandate — if the architecture needs different seams, say so explicitly and map area-* → comp-* so the chain still resolves.

How the OQ-4 block was resolved in the build: the agent doesn't refuse the run, it splits it. Module boundaries, data model, API surface and comp-* assignment all derive from functional requirements and

proceed. Stack, hosting, scaling, auth and caching are held as ADRs marked **Proposed – blocked by 0Q-n**, each naming the NFR answer that would settle it, and the missing NFRs are routed back to 01 as a PRD amendment. Provisional answers given in conversation are recorded as **Assumptions**, never as requirements. This is rule 7 ("block the narrowest slice") applied.

Also built in: ADRs are **append-only** (superseded, never edited or renumbered) and module slugs are permanent (retired, never renamed or reused) — the same id discipline the PRD applies to **REQ-**.

08 — Change Request & Bug Triage Agent **next to build**

Plan change I'm proposing. Originally this sat at the end of the pipeline. It should come early. It is the agent the entire label schema was designed to serve, and its routing/impact half is fully testable against the existing ACRI backlog *before any code exists*. Building it early validates — or falsifies — the whole traceability investment while it's still cheap to change.

Replaces	Support/triage lead deciding what a new request actually is
Input	A raw bug report or change request in the client's own words
Output	A triage verdict + a Jira Bug or CR ticket linked to affected stories; for a scope change, an amendment request routed to 01
Mints	Bug/CR tickets, cr-<nnn> labels
Inherits	Everything — this is the consumer of the whole chain
Feeds	01 (PRD amendment if scope changes), 02 (re-sync if requirements moved), 05 (fix work)

Core logic:

1. Extract area/keywords → JQL against the label schema → candidate stories.
2. Compare the report against each candidate's **Acceptance Criteria**.
 - Violates an existing AC → **Bug**. File against that story, cite the AC.
 - Not covered by any AC, and named in an **Out of Scope** section → **Change Request**. Route to 01 for a PRD amendment; do not quietly implement it.
 - No matching story at all → flag as a coverage gap, escalate to human.
3. Report blast radius: which **comp-*** modules and which other stories share them.

This is why **Out of Scope** and Given/When/Then criteria are mandatory upstream — they are the inputs to this decision.

04 — UX / Design Agent optional, parallel to 03

Input	Confirmed PRD + stories
Output	4. Design/ — user flows, wireframes/screen specs, component inventory, design tokens
Inherits	persona slugs, area-* , glossary
Feeds	05 (what the UI should be), 07 (visual/UX acceptance)

Skip for API-only or backend products. For the perfume storefront it matters — OQ-3 (no competitor/UX direction given) is its blocker.

05 — Implementation Agent

Replaces	Developer picking up a ticket
Input	One Jira story + architecture doc (+ design spec if 04 exists)
Output	Working code on a branch, one branch/PR per story
Inherits	US-xxx (branch name + commit trailer), comp-* (where code goes), AC (what "done" means)
Feeds	06, 07
Rules	One story per branch. Commits reference US-012 . Implements the AC and nothing beyond it — extra scope goes back to 08 as a CR. Does not invent requirements.

06 — Code Review Agent

Input	A PR from 05 + its story + architecture doc
Output	Review verdict: correctness bugs, architecture-drift flags, AC-coverage check
Rules	Verifies the PR satisfies <i>every</i> AC and stays inside the module boundaries 03 defined. Architecture drift is a finding, not a preference.

07 — QA / Test Agent

Input	A story's Acceptance Criteria + the implementation
Output	Test cases (one per AC, unhappy paths included) + execution results
Inherits	US-xxx , AC verbatim
Rules	Each test names the US-/REQ- id it verifies, so a failure points straight back up the chain. NFR criteria (once OQ-4 is answered) get their own tests.

09 — Release Agent

Input	Merged stories since last release
Output	Release notes grouped by area-* , changelog citing REQ-/US- ids, deploy checklist
Rules	Hard approval gate — this touches prod. Notes are written for the client, in Glossary vocabulary, not ticket-speak.

6. Suggested build order

Order	Agent	Why here
+	03-Architecture	Built. NFR-dependent decisions stay Proposed until OQ-4 is answered.
1	08 CR / Bug Triage	Validates the traceability design while it's cheap to change; testable on today's backlog with zero code.
3	05 Implementation	The first agent that produces running software.
4	07 QA / Test	Pairs with 05; AC → tests is mechanical once stories are good.
5	06 Code Review	Valuable once there's enough code for drift to be real.
6	04 UX / Design	Slot in before 05 if the product is UI-heavy (the perfume store is).
7	09 Release	Last — needs something to ship.

7. File layout

<code>.claude/agents/<name>.md</code>	executable agent (frontmatter + prompt)
<code>agents/<nn>-<name>/agent.md</code>	human-readable spec
<code>agents/<nn>-<name>/usage.md</code>	how to run it
<code>agents/AGENT-DESIGN-GUIDE.md</code>	design rules + rationale (read before adding an agent)
<code>agents/README.md</code>	pipeline index
<code>ROADMAP.md</code>	this file
1. PRD/	01 output
2. User Stories/	02 output (draft + index cache)
3. Architecture/	03 output (architecture.md + component-index.md)
4. Design/	04 output (not yet created)

Convention when adding an agent: write the executable prompt in `.claude/agents/`, the human spec in `agents/<nn>-<name>/`, then update `agents/README.md` and this roadmap.

8. Checklist for designing the next agent

1. **One transformation** — one input document, one output artifact, stated in one line.
2. **What does it NOT do?** Write the exclusions into the prompt explicitly.
3. **How does its output get found later?** Which ids does it inherit, which does it mint, what query retrieves them?
4. **What happens on the second run?** Dedupe key, and what tells it which parts changed.
5. **What happens when its input changes underneath it?** Versioned input, changed ids, retired ids.
6. **Where's the approval gate?** Anything shared or irreversible. Nothing is ever deleted.
7. **When does it escalate?** And does it block the narrowest slice possible, not the whole run?
8. **Verify the tool schemas first.**

9. Decisions still open

Item	Needs	Blocks
OQ-4 — no NFRs captured (performance, security, accessibility, browsers, data retention)	Client answers; PRD amended to v1.2 adding REQ-016+	03 Architecture, sound AC everywhere
OQ-1 — courier/logistics partner	Client decision	REQ-009 story (order tracking)
OQ-2 / OQ-3 — success metrics, UX direction	Client input	04 UX; post-launch measurement
Should ACRI be the permanent home?	It's named "Autonomous change request integration" — a real perfume project would want its own Jira project	Nothing yet; migration later means re-running 02 (ids survive, Jira keys change)
Depends-On as real Jira links	One pass of createIssueLink over the 21 stories	Nothing critical; nice for sprint planning